

O Apocalipse do Delivery

Vocês são os times de Arquitetura de Testes e SRE da plataforma **EntregasJá**. O sistema opera em microsserviços usando Node.js/Express e interage com um banco de dados relacional e um gateway bancário parceiro externo.

A diretoria está em pânico: o **Black Friday** está chegando e há uma suspeita crítica de que, se o gateway parceiro ou o cache falharem, o sistema sofrerá um efeito cascata catastrófico, travando por exaustão de threads e derrubando toda a aplicação.

O objetivo do seu grupo é blindar o microsserviço de **Processamento de Pedidos e Checkout**, redesenhando o código de forma limpa, garantindo cobertura contra mutantes, tratando o comportamento com BDD e provando que a arquitetura sobrevive ao caos através de testes de performance e injeção de falhas na infraestrutura.

1. Fases do Trabalho e Artefatos a Produzir

O trabalho deve ser realizado em **grupos** e cobrirá as seguintes frentes obrigatórias:

Fase 1: Análise Estrutural, Complexidade e Métricas de Estimativa

Antes de tocar no código, vocês precisam auditar o componente legado fornecido de Checkout.

- **Mapeamento de Fluxo:** Desenhar o Grafo de Fluxo de Controle do método principal de processamento de pedidos.
- **Complexidade Ciclomática:** Calcular matematicamente o $V(G)$ do componente para descobrir o número mínimo de caminhos independentes que precisam ser cobertos.
- **Métricas e Estimativas de Teste:** Montar um documento formal de estimativa (utilizando a técnica de base que discutimos em sala, como pontos de caso de teste ou análise de pontos de função adaptada para testes), detalhando o esforço em horas/homem e recursos necessários para testar esta funcionalidade por completo.

Fase 2: Redesenho com TDD, BDD e Padrões de Projeto

O código atual é uma colcha de retalhos e acoplamento rígido.

- **BDD (Gherkin):** Escrever a especificação viva em arquivos `.feature` contendo os cenários de sucesso e caminhos infelizes (timeout do gateway, cartão recusado, erro de infraestrutura) seguindo a estrutura *Dado-Quando-Então*.
- **Ciclo TDD:** Implementar a solução a partir dos testes (Test-First), respeitando o ciclo *Vermelho-Verde-Refatore*.
- **Test Patterns & Clean Code:** Proibir o cheiro de código de *Obscure Setup*. Vocês devem obrigatoriamente aplicar o padrão **Data Builder** ou **Object Mother** para a fabricação modular das faturas/pedidos. As dependências de e-mail e pagamento devem usar **Stubs** para cenários de estado e **Mocks** para asserção de comportamento (ex: garantir que o e-mail de confirmação é disparado apenas em caso de sucesso).
- **Refatoração Baseada em Padrões:** Identificar *Test Smells* no código legado e aplicar refatorações clássicas do Fowler, como *Extract Method*, *Introduce Parameter Object* ou *Replace Conditional with Polymorphism* para eliminar os blocos excessivos de `if/else`.

Fase 3: Teste de Mutação (A Prova de Fogo do Código)

Não basta ter 100% de cobertura de linhas (o que já provamos que não garante eficácia).

- Vocês devem rodar uma ferramenta de Teste de Mutação (como Stryker.js para o ecossistema Node/JS).

- A meta obrigatória é atingir um **Mutation Score mínimo de 80%**. Se mutantes sobreviverem (como trocas de operadores condicionais ou eliminação de comandos), vocês precisarão enriquecer a suíte de testes unitários/integração para eliminá-los.

Fase 4: Engenharia do Caos e Testes de Desempenho (SRE)

Agora tiramos o isolamento do código e testamos o sistema sob condições extremas em ambiente de homologação simulado.

- **Teste de Carga e Estresse (k6 ou Autocannon):** Criar scripts de teste simulando a volumetria da Black Friday (padrão *ramp-up*, *steady*, *ramp-down*). Definir rigidamente os limites de **SLI/SLO** da aplicação (ex: p95 da latência das requisições deve se manter abaixo de 5 segundos; taxa de erro abaixo de 5%).
- **Injeção de Falhas no Ambiente (Toxiproxy):** Durante a execução do teste de carga máxima do k6, vocês devem injetar "tóxicos" na infraestrutura simulando os seguintes desastres:
 1. **O Thundering Herd (Manada Estourada):** Dar um *flush* completo ou derrubar o nó de cache repentinamente com 10.000 requisições simultâneas para avaliar se o banco de dados sobrevive usando mecanismos de *Backoff* e *Jitter*.
 2. **Gateway Lento:** Adicionar 5000ms de latência na comunicação com a API de pagamento parceira.

2. Entrega em Vídeo (Apresentação Prática)

Além de submeter o repositório Git com o código, os relatórios de mutação e os gráficos de desempenho do k6, cada grupo gravará um **vídeo de 8 a 12 minutos** simulando uma reunião de revisão técnica com os *stakeholders*. No vídeo, o grupo deve:

1. Explicar brevemente as estimativas e os resultados da análise ciclômática.
2. Mostrar a especificação em Gherkin e o código refatorado rodando e passando de forma linear no pipeline.
3. **A melhor parte:** Mostrar a tela dividida com o k6 gerando carga massiva e o exato momento em que o Toxiproxy injeta o caos na rede, destacando se as métricas de SLO foram mantidas ou violadas e como o sistema se comportou.

Item de Avaliação	Descrição Detalhada	Pontuação
Fase 1: Análise estrutural e métricas*	Correção do Grafo de Fluxo, cálculo correto da Complexidade Ciclômática e maturidade no documento de estimativa de esforço de teste.	2,0 pts
Fase 2: TDD, BDD e Padrões de Código*	Escrita adequada de cenários Gherkin , aplicação correta de Test Patterns (Builders, Stubs e Mocks) , eliminação de Test Smells e qualidade da refatoração.	3,0 pts
Fase 3: Teste de Mutação*	Eficácia comprovada dos testes unitários através do Mutation Score $\geq 90\%$ e justificativa técnica de possíveis mutantes equivalentes.	2,0 pts
Fase 4: Caos e Performance (SRE)*	Configuração realista do k6/Autocannon , definição correta de thresholds de SLO , execução do cenário de Thundering Herd e prova de Degradação Graciosa com cálculo de MTTR.	2,5 pts

Qualidade da Entrega e Vídeo	Clareza técnica na apresentação, capacidade de defender as decisões arquiteturais tomadas e demonstração visual fluida dos experimentos rodando.	2,0 pts
------------------------------	--	---------

* Todos os itens serão avaliados mediante a entrega do vídeo de demonstração

Repositório base: <https://github.com/CleitonSilvaT/o-apocalipse-do-delivery>

Dicas:

O segredo da engenharia de software distribuída é dar **passos de bebê (baby steps)**. Se o código de vocês estiver frágil no nível micro (try...catch mal estruturado), a injeção de caos no nível macro vai derrubar o sistema imediatamente e o k6 vai pintar a tela de vermelho com erros 5xx.

Preparem as ferramentas, configurem seus proxies e que vença o time com a arquitetura mais resiliente!

Bom trabalho!